J U L I A

Управление потоком · ошибки · модули · параметрические типы · параллелизм

Дмитриенко Яна · Шабанова Арина

Группа 5030102/30401

Курс «Новые языки программирования»

Управление потоком

```
x = 10

if x > 0
    println("positive")
elseif x == 0
    println("zero")
else
    println("negative")
end
```

Тернарный оператор

status = x > 0 ? "positive" : "non-positive"

Логические сокращения

condition && action() # если true → выполнить
condition || fallback() # если false → выполнить

Важно

Условия в if / тернарном операторе должны иметь тип Bool.
Числа не используются как true/false автоматически.

Циклы

for

```
for i in 1:5
    println(i)
end
```

while

```
i = 1
while i <= 5
    println(i)
    i += 1
end
```

break и continue

break — немедленный выход из цикла
continue — переход к следующей итерации
Работают и в for, и в while

Диапазоны — итерируемые объекты

1:5 — это Range, не массив
Ленивое вычисление — память не выделяется
for i in 1:5 использует протокол итерации

Вложенные циклы

for i in 1:3, j in 1:3 — компактный синтаксис

Итерация в Julia

for работает через протокол итерации — не только с массивами

```
for value in [10, 20, 30]
    println(value)
end

# Эквивалент через iterate():
itr = iterate(collection)
while itr !== nothing
    (value, state) = itr
    println(value)
    itr = iterate(collection, state)
end
```

Протокол итерации

iterate(collection)
→ (value, state) или nothing

iterate(collection, state)
→ следующая пара или nothing

Любой тип, реализующий iterate(), работает в for

Главная мысль

for в Julia универсален: массивы, Dict, строки, Range, Set — всё итерируемо через единый протокол iterate()

Рекурсия

```
function factorial(n)
    n <= 1 && return 1
    n * factorial(n - 1)
end

println(factorial(5))  # 120
```

```
120
```

Базовый случай

n <= 1 && return 1 — условие остановки

Рекурсивный вызов

n * factorial(n - 1) — сведение к меньшей задаче

Ограничение глубины

Для очень глубокой рекурсии лучше итеративный подход
Julia не гарантирует tail-call optimization

Замыкания

```
function make_counter()
    count = 0
    return function ()
        count += 1
        count
    end
end

counter = make_counter()
println(counter())  # 1
println(counter())  # 2
println(counter())  # 3
```

```
1
2
3
```

Что происходит

make_counter() создаёт переменную count
Внутренняя функция «захватывает» count
Каждый вызов counter() увеличивает count
В Julia не нужен global/nonlocal

Суть замыкания

Функция + окружение, где она создана
Переменные внешней области доступны
Даже после завершения внешней функции

Исключения

```
try
    value = parse(Int, "abc")
catch e
    println("Ошибка: ", typeof(e))
finally
    println("Готово")
end
```

```
Ошибка: ArgumentError
Готово
```

Типичные исключения Julia

ArgumentError — неверный аргумент
BoundsError — выход за границы
DomainError — вне области определения
MethodError — нет подходящего метода
InexactError — неточное преобразование

Конструкция

try → код, который может выбросить исключение
catch → обработка ошибки
finally → выполняется всегда

Создание ошибок и @assert

throw

```
function divide(a, b)
    b == 0 && throw(ArgumentError("b ≠ 0"))
    a / b
end
```

error()

```
error("Something went wrong")
# Выбрасывает ErrorException
```

@assert

```
@assert x > 0 "x must be positive"
# Если x ≤ 0 → AssertionError
```

throw / error / @assert

throw(expr) — выбрасывает любой объект
error(msg) — выбрасывает ErrorException
@assert cond msg — проверка предположений
не заменяет обработку ошибок!

Разница

Исключения — реакция на ошибочные ситуации
@assert — проверка инвариантов программы
В production @assert может быть отключён

Модули и пространства имён

```
module Geometry

export area

area(r) = π * r^2

end
```

```
using .Geometry
println(area(2))  # 12.566...
```

```
12.566...
```

module

Создаёт отдельное пространство имён
Имена внутри не видны снаружи автоматически
Организация крупного проекта

export

Определяет, какие имена доступны через using
Неэкспортированные нужны с квалификацией
Geometry.circumference — полный путь

using .Module

Точка — модуль в текущем scope
using без точки — из установленных пакетов
Помогает избежать конфликтов имён

using, import и export

using ModuleName

Привносит ВСЕ export-имена в scope
Можно вызвать напрямую: area(2)

using ModuleName: name1, name2
— выбрать конкретные имена

export

Указывает «публичный API» модуля
export name1, name2

using-импорт видит только export-имена
import — видит все имена

import ModuleName

Имя доступно ТОЛЬКО с квалификацией
Module.func() — нужен префикс

import ModuleName: name
— конкретное имя без квалификации
— можно добавлять методы!

Ключевое отличие

using — импорт + привнесение в scope
import — нужна квалификация

import нужен для extension:
добавление методов к чужим функциям

Пакеты и Pkg

```
using Pkg
Pkg.add("Example")
```

Pkg mode

```
]          # войти в Pkg mode
add Example
status
```

Воспроизводимость

julia --project=.
Pkg.instantiate()
Одинаковое окружение у всех

Project.toml

Прямые зависимости проекта
Совместимость версий [compat]
UUID, имя пакета, инфо об окружении

Manifest.toml

Точные версии ВСЕХ зависимостей
Гарантия воспроизводимости окружения
Pkg.instantiate() — установить по Manifest

Важно для команды

Manifest.toml гарантирует одинаковые версии у всех разработчиков

Параметрические типы

```
struct Point{T}
    x::T
    y::T
end

p1 = Point(1, 2)        # Point{Int64}
p2 = Point(1.0, 2.0)    # Point{Float64}

typeof(p1)  # Point{Int64}
typeof(p2)  # Point{Float64}
```

Point{Int64} ≠ Point{Float64}

Каждое значение T — отдельный конкретный тип
Это разные типы — не один «универсальный»
Компилятор специализирует код для каждого T

Отличие от C++ / Java

C++ templates — кодогенерация при компиляции
Java generics — type erasure, один класс
Julia — полноценные типы во время выполнения
+ специализация через JIT

Параметрические методы

```
function distance(p::Point{T}) where {T<:Real}
    sqrt(p.x^2 + p.y^2)
end

distance(Point(3, 4))     # 5.0
distance(Point(3.0, 4.0)) # 5.0
```

```
5.0
```

where {T<:Real}

where задаёт параметр типа T
T <: Real ограничивает допустимые типы
Point{String} не пройдёт ограничение

Связь с multiple dispatch

Диспетчеризация + параметрические типы
= гибкий generic-код
Компилятор создаёт оптимизированный вариант для каждого конкретного T

Работа с файлами

запись

```
open("data.txt", "w") do io
    println(io, "Hello, Julia!")
end
```

чтение

```
text = read("data.txt", String)
println(text)  # "Hello, Julia!"
```

Конструкция do

open("file", "w") do io ... end
Файл закрывается автоматически при выходе
Гарантирует управление ресурсом (RAII-like)
Аналог with в Python

Без do-блока

```
io = open("data.txt", "w")
println(io, "text")
```

Другие операции I/O

readline(), eachline(), readdir() — построчное чтение и каталоги
open() с "a" — дозапись, "r+" — чтение+запись
Стандартные потоки: stdin, stdout, stderr

Конкурентность: Tasks

@async

```
task = @async begin
    sleep(1)
    println("done")
end

wait(task)
```

Task — кооперативная задача

@async планирует задачу в текущем потоке
Переключение при yield-точках (sleep, I/O)
Это конкурентность, НЕ параллелизм

wait(task)

Дождаться завершения задачи

Concurrency ≠ Parallelism

Конкурентность — задачи чередуются, могут быть на одном ядре
Параллелизм — задачи выполняются одновременно на разных ядрах
@async — конкурентность; Threads.@threads — параллелизм

Потоки: Threads

@threads

```
using Base.Threads

println(nthreads())

Threads.@threads for i in 1:8
    println("i=$i on thread $(threadid())")
end
```

Запуск с потоками

julia --threads auto script.jl
julia -t auto script.jl

Свойства

Порядок вывода НЕ гарантирован
Итерации распределяются по потокам
Threads.@spawn — более гибкий вариант

Синхронизация

Общие изменяемые данные требуют синхронизации!
ReentrantLock, Threads.Atomic, SpinLock — инструменты Julia
Обычный push! в общий массив из разных потоков — небезопасно

Управление памятью

```
a = [1, 2, 3]
push!(a, 4)  # мутация — изменяет a

# ! — соглашение: функция изменяет аргумент
```

Garbage Collector

Память управляется автоматически
GC освобождает недостижимые объекты
Пользователь не вызывает free/delete

Аллокации и производительность

Мутация часто быстрее аллокации
Избегание лишних аллокаций — ключ к скорости
Компилятор оптимизирует размещения

Соглашение !

! в имени функции — предупреждение о мутации
push!, sort!, splice! — изменяют аргумент
! не включает специальный режим языка

Mutability

struct — неизменяемый тип; mutable struct — изменяемый
Immutable объекты могут оптимизироваться компилятором (stack allocation)

Итоги

Исключения

try/catch/finally + @assert

Файловый I/O

open() do / read / write

Память и GC

Автоматическое управление / !

Финальная мысль

Julia сочетает динамический высокоуровневый синтаксис с системой типов и моделью выполнения, ориентированной на производительные вычисления.

Спасибо за внимание! Вопросы?